

Sistemas Distribuídos

- Introdução
- Exemplos de Sistemas Distribuídos
- Características suportadas pelos Sistemas Distribuídos
- Modelos Arquitecturais
- Modelos Fundamentais

Programação Distribuída / José Marinho

Introdução

- Generalização das Redes de computadores
- Os componentes de um sistema distribuído
 - Situa-se em dispositivos ligados em rede
 - Interagem apenas via mensagens
- Os domínios de aplicação são inúmeros
- O objectivo é sempre a troca de dados e a partilha de recursos (e.g., periféricos, bases de dados e capacidade de processamento)

2

Programação Distribuída / José Marinho

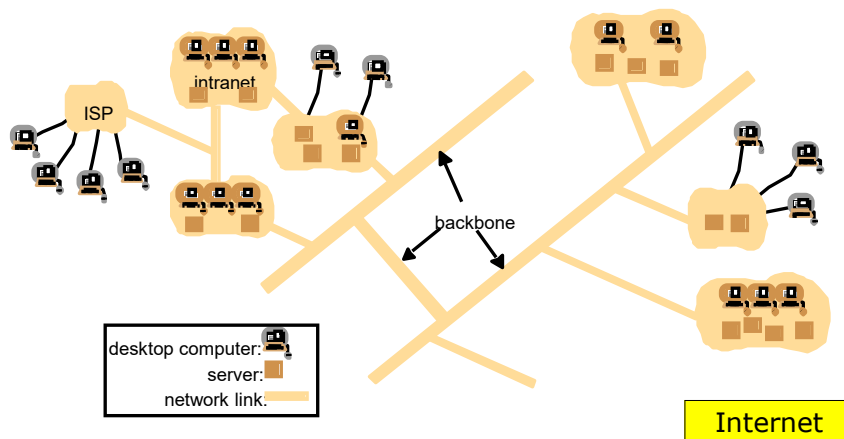
Introdução

- Consequências desta abordagem distribuída
 - Concorrência
 - Ausência de relógio global
 - Independência das falhas
- Existem vários aspectos, técnicas e algoritmos distintos a considerar no projecto de sistemas distribuídos

3

Programação Distribuída / José Marinho

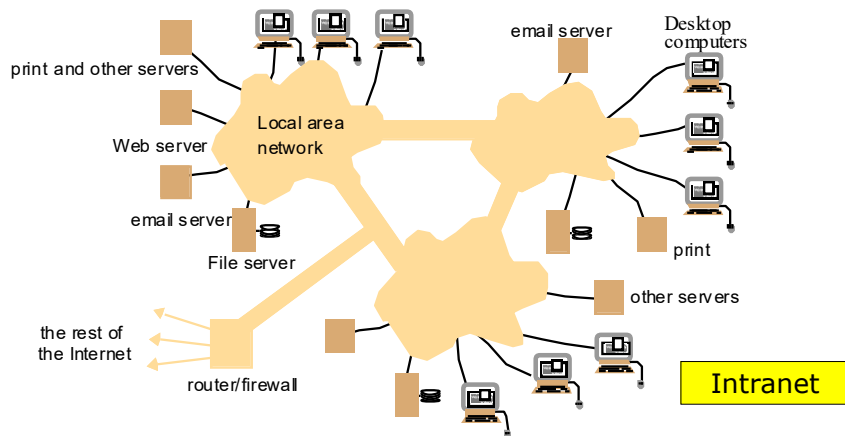
Exemplos de Sistemas Distribuídos



4

Programação Distribuída / José Marinho

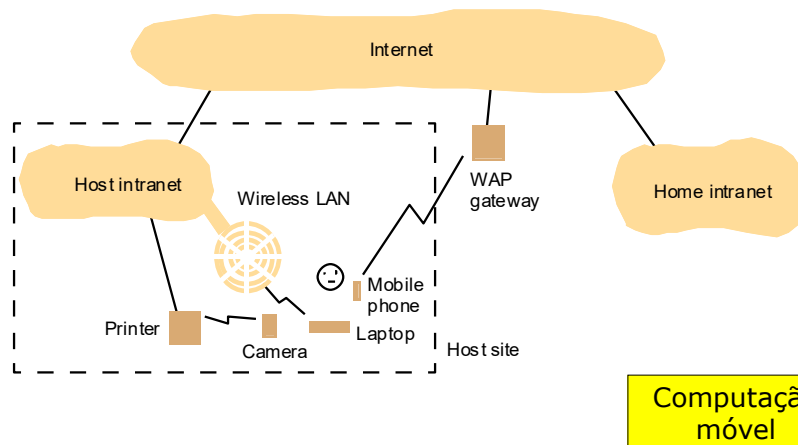
Exemplos de Sistemas Distribuídos



5

Programação Distribuída / José Marinho

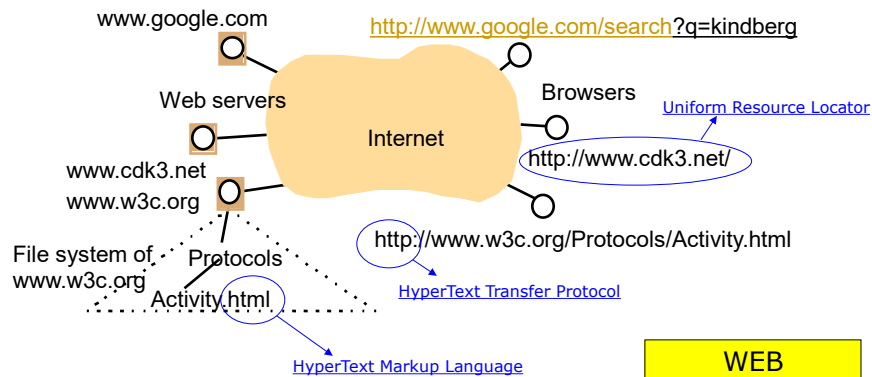
Exemplos de Sistemas Distribuídos



6

Programação Distribuída / José Marinho

Exemplos de Sistemas Distribuídos



7

Programação Distribuída / José Marinho

Características Pretendidas

- **Heterogeneidade**
Ao nível das redes, do *hardware*, dos sistemas operativos e das linguagens de programação
- **Abertura**
- **Segurança**
Confidencialidade, integridade e disponibilidade
- **Escalabilidade**
Manter o nível de desempenho e as funcionalidades quando aumentam os recursos e/ou os utilizadores

8

Programação Distribuída / José Marinho

Características Pretendidas

- Tolerância a falhas
- Concorrência

Acesso de um mesmo recurso por vários clientes em simultâneo
- Transparência

Abstrair o utilizador de vários aspectos como o tipo de acesso (local e remoto), a localização dos recursos, os mecanismos de replicação e a detecção/recuperação de falhas

9

Programação Distribuída / José Marinho

Introdução aos Modelos

- Modelos
 - Descrição abstracta e consistente de um aspecto relevante dos sistemas distribuídos (e.g., modelo cliente/servidor)
 - Modelos arquitecturais e fundamentais
- Modelos arquitecturais
 - Divisão das responsabilidades pelos diversos componentes e respectiva distribuição pelos computadores de uma rede
 - Cliente/servidor
 - Pares de processos
 - Cliente/servidor modificado

10

Programação Distribuída / José Marinho

Introdução aos Modelos

- Modelos fundamentais
 - Analisar os Sistemas Distribuídos em relação às características básicas presentes em qualquer arquitectura
 - Modelo de interacção
Analisa o desempenho e dificuldade de estabelecer limites temporais na interacção entre os vários elementos
 - Modelo de falha
Especificação precisa das falhas que podem ocorrer nos processos e canais de comunicação
 - Modelo de segurança
Conceito de canal seguro e referência às possíveis ameaças aos processos e canais de comunicação

11

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Objectivos
 - Identificação dos vários componentes
 - Divisão das responsabilidades pelos diversos componentes e respectiva distribuição pelos computadores de uma rede
 - Preocupações de fiabilidade, gestão, adaptabilidade e custos adequados

12

Programação Distribuída / José Marinho

Modelos Architecturais

- Aspectos a considerar
 - Localização dos diversos componentes nas redes de computadores
 - Definição de padrões úteis para a distribuição e carga dos dados
 - Interações existentes entre os vários componentes (papéis funcionais e padrões de comunicação).

13

Programação Distribuída / José Marinho

Modelos Architecturais

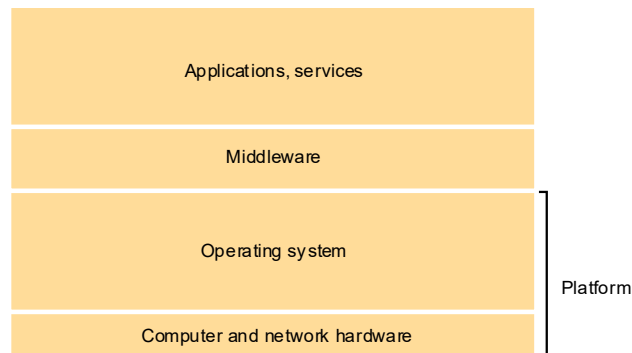
- Três tipos de processos
 - Servidores
 - Clientes
 - Pares de processos
 - Variações à classificação anterior
 - Exemplo: recurso a código móvel

14

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Camadas de *Software*



15

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Camada de *Middleware*

- Oculta a heterogeneidade
- Oferece um modelo de programação abstracto
 - Invocação remota de métodos
 - Comunicação entre grupos de processos
 - Transmissão em tempo real
 - Replicação de dados partilhados
 - ...
- Exemplos: Java RMI, RPC da Sun, Corba, MPI e .Net Remoting

16

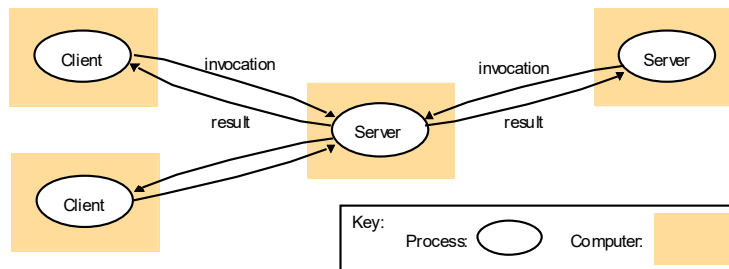
Programação Distribuída / José Marinho

Modelos Arquitecturais

- Modelos de base

- Cliente/servidor

Um servidor pode ser cliente de outro serviço



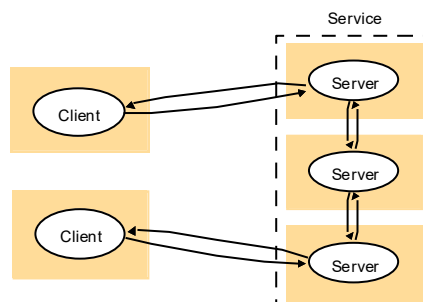
17

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Múltiplos servidores

Baseados em objectos distribuídos (e.g., WEB) ou replicados para efeitos de melhoria de desempenho, de disponibilidade e de tolerância a falhas

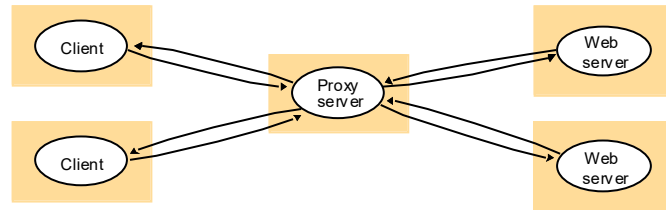


18

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Servidores Proxy

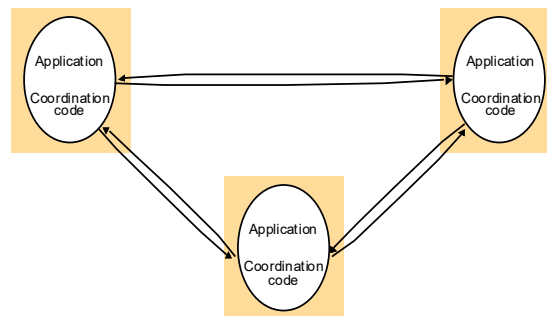


19

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Processos Pares



20

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Variações aos modelos de base
 - Computadores de rede
 - Gestão local simplificada
 - Carregamento do S.O. e das aplicações de um servidor de ficheiros remoto
 - Pode possuir disco local para ser usado como *cache*

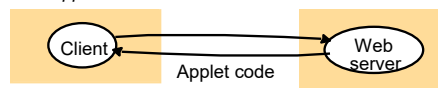
21

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Código móvel
 - Problemas de segurança
 - Exemplo: *applets*

1- Carregamento da *Applet*



2- Utilização da *Applet*



22

Programação Distribuída / José Marinho

Modelos Arquitecturais

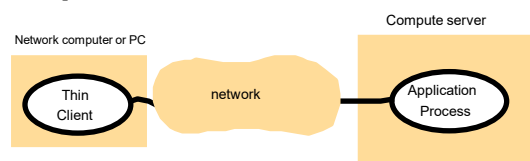
- Agentes móveis
 - Programa em execução (código + dados) que “viaja” de computador em computador
 - Por exemplo, em bases de dados, evita a transferência remota de grandes quantidades de dados
 - Problemas de segurança para os *hosts* e próprios agentes
 - Aplicabilidade limitada

23

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Clientes “finos”
 - Camadas de *software* que oferecem uma interface gráfica aos utilizadores locais, sendo a aplicação executada remotamente
 - Semelhante a computadores de rede, mas sem processamento local
 - Poucos requisitos ao nível do *hardware* necessário
 - A largura de banda é um factor crítico, mas, com a generalização dos acessos de banda larga, a descentralização de recursos (*software* e *hardware*) tem sido uma prática cada vez mais comum (conceito de *Cloud* na Internet)



24

Programação Distribuída / José Marinho

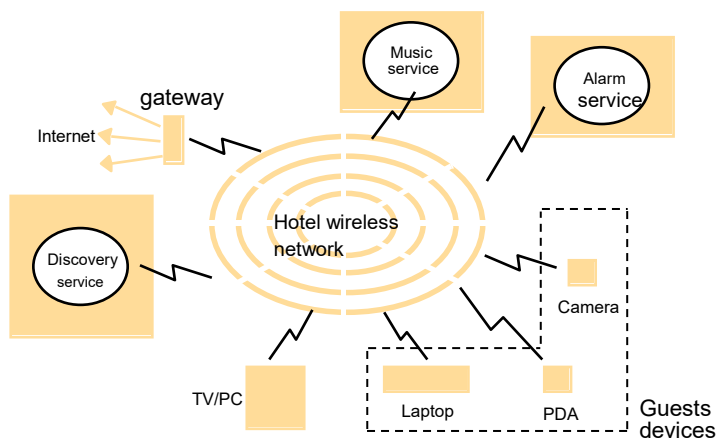
Modelos Arquitecturais

- Dispositivos móveis e redes espontâneas
 - Ligações simplificadas às redes locais
 - Fácil integração com os serviços locais
 - Levanta várias questões
 - Endereçamento
 - Conectividade
 - Segurança/privacidade
 - Necessita de um mecanismo do tipo *discovery service* constituído por duas interfaces/serviços distintos: *registration service* e *lookup service*

25

Programação Distribuída / José Marinho

Modelos Arquitecturais



26

Programação Distribuída / José Marinho

Modelos Arquitecturais

- Objectos remotos e Interfaces remotas
 - Os processos podem ser encapsulados em objectos/serviços cujas referências são passadas aos clientes para invocação remota das interfaces (e.g., Corba e Java RMI)
 - Interface remota: conjunto de funções disponibilizadas para interagir com um processo remoto
 - Existe vários tipos de objectos/serviços remotos oferecidos pelas soluções de *middleware* existentes (*singleton*, *single-call*, *client-activated*, *stateful*, *stateless*, etc.)

27

Programação Distribuída / José Marinho

Modelos Arquitecturais

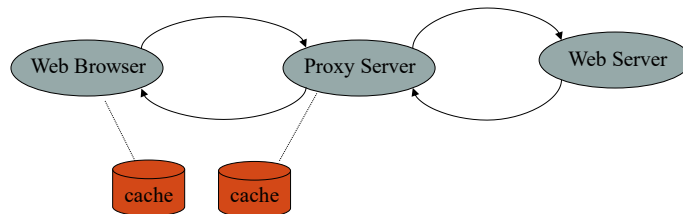
- Requisitos de projecto
 - Desempenho
 - Responsividade
Depende de muitos factores tais como a quantidade de dados e de camadas de *hardware* e *software*
 - *Throughput*
Capacidades de processamento e de transferência dos dados
 - Equilíbrio das cargas de computação
Evitar acessos concorrentes aos mesmos recursos (e.g., com *applets*)

28

Programação Distribuída / José Marinho

Modelos Arquitecturais

Exemplo: recurso a *proxies* e caches para melhorar a responsividade



29

Programação Distribuída / José Marinho

Modelos Arquitecturais

- QoS (Qualidade de Serviço)
 - Confiabilidade (modelo de falha)
 - Segurança (modelo de segurança)
 - Desempenho (modelo de interacção)
 - Adaptabilidade
 - Certas aplicações possuem dados com requisitos temprais (*time-critical*)

30

Programação Distribuída / José Marinho

Modelos Arquitecturais

- *Caching* e replicação
 - Resposta aos problemas de QoS e de desempenho
- Tolerância a falhas
 - Continuar a operar correctamente na presença de falhas de *hardware*, de *software* e de comunicação
 - Conseguído à custa de redundância

31

Programação Distribuída / José Marinho

Modelos Fundamentais

- Tornam explícitos todos os pressupostos relevantes do sistema a modelizar
- Definem, em termos gerais, o que é possível e impossível
- Ajudam a reflectir e a discutir várias questões
 - Interacção entre processos
 - Falhas
 - Segurança

33

Programação Distribuída / José Marinho

Modelos Fundamentais

- Modelo de interacção
 - Os processos interagem de um modo complexo
 - Comportamentos e estados descritos por algoritmos distribuídos
 - Dificuldade de previsão dos ritmos de execução de cada processo e do tempo de transmissão das mensagens
 - Dificuldade de descrição/enumeração de todos os estados possíveis

34

Programação Distribuída / José Marinho

Modelos Fundamentais

- Desempenho dos canais de comunicação
 - Latência
 - Largura de banda
 - *Jitter*

35

Programação Distribuída / José Marinho

Modelos Fundamentais

- Relógios de computador e temporização de eventos
 - Cada máquina possui o seu relógio local
 - Cada processo pode associar tempos aos seus eventos
 - É praticamente impossível os diversos relógios estarem sincronizados
 - É possível, com receptores adequados, ler a hora de um GPS com precisão de $1\ \mu\text{s}$ (solução onerosa e impossível de usar dentro de edifícios)
 - A solução mais viável baseia-se na sincronização através de mensagens enviadas a partir de um computador (atenção aos diversos tempos de atraso!)

36

Programação Distribuída / José Marinho

Modelos Fundamentais

- Sistemas distribuídos síncronos
 - São conhecidos limites inferiores e superiores para o tempo de execução de cada processo e para o tempo de transferência das mensagens
 - O desvio relativo dos relógios locais é conhecido
 - Permite o recurso a *timeouts* para detectar falhas de processos
 - Este modelo dificilmente corresponde a sistemas reais, mas é útil na previsão de comportamentos

37

Programação Distribuída / José Marinho

Modelos Fundamentais

- Sistemas distribuídos assíncronos
 - A Internet como exemplo
 - Cada instrução num processo pode ter uma duração variável, não sendo possível definir limites
 - Uma mensagem pode ser recebida depois de um período arbitrário
 - O desvio dos relógios é arbitrário
 - Não se pode assumir valores para os tempos de execução
 - A maior parte dos sistemas actuais são assíncronos

38

Programação Distribuída / José Marinho

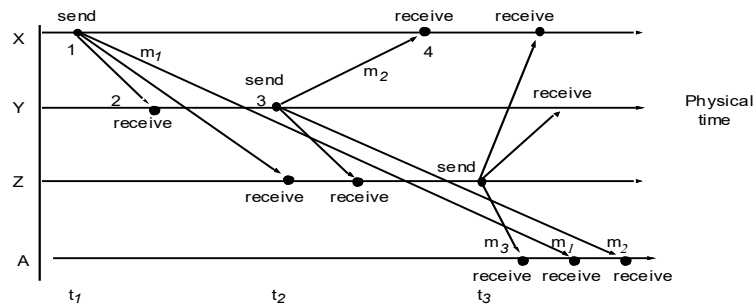
Modelos Fundamentais

- Ordenação dos eventos
 - Por vezes é necessário conhecer a ordem de ocorrência de vários eventos em processos distintos (e.g., junção ordenada de *logs* de vários processos numa aplicação distribuída)
 - Um solução baseada em relógios sincronizados consiste em rotular cada mensagem com informações temporais
 - Quando os relógios não podem ser sincronizados, recorre-se ao modelo de tempo lógico

39

Programação Distribuída / José Marinho

Modelos Fundamentais



40

Programação Distribuída / José Marinho

Modelos Fundamentais

- Modelo de falha
 - Define o modo como as falhas podem ocorrer nos processos e canais de comunicação, na sequência de erros, de modo a perceber as suas consequências
 - Falhas por omissão de processos
 - É mais fácil de recuperar quando o *crash* é limpo (i.e., ou funciona correctamente ou pára)
 - Os outros processos detectam a ocorrência por não ser repetidamente recebida qualquer resposta a mensagens enviadas
 - *Fail-stop*, *Send-omission* e *Receive-omission*

41

Programação Distribuída / José Marinho

Modelos Fundamentais

- Falhas por omissão de comunicação
 - O canal de comunicação falha quando não transporta a mensagem pretendida até ao *buffer* do destino
 - Perda de mensagens provocada por, por exemplo, falha dos *buffers* no destino ou *gateways* intermédias e eliminação devido a *bits* invertidos
- Falhas arbitrárias ou bizantinas
 - Omissões e inserções arbitrárias de passos
 - Produção ou inserção de valores/dados/resultados errados
 - Difíceis de detectar
 - Corrupção e entrega duplicada de mensagens (certos protocolos de comunicação detectam este tipo de falhas)

Modelos Fundamentais

<i>Class of failure</i>	<i>Affects</i>	<i>Description</i>
Fail-stop	Process	Process halts and remains halted. Other processes may detect this state.
Crash	Process	Process halts and remains halted. Other processes may not be able to detect this state.
Omission	Channel	A message inserted in an outgoing message buffer never arrives at the other end's incoming message buffer.
Send-omission	Process	A process completes a <i>send</i> , but the message is not put in its outgoing message buffer.
Receive-omission	Process	A message is put in a process's incoming message buffer, but that process does not receive it.
Arbitrary (Byzantine)	Process or channel	Process/channel exhibits arbitrary behaviour: it may send/transmit arbitrary messages at arbitrary times, commit omissions; a process may stop or take an incorrect step.

Modelos Fundamentais

- Falhas temporais
 - Em sistemas síncronos
 - Limite temporal ultrapassado

<i>Class of Failure</i>	<i>Affects</i>	<i>Description</i>
Clock	Process	Process's local clock exceeds the bounds on its rate of drift from real time.
Performance	Process	Process exceeds the bounds on the interval between two steps.
Performance	Channel	A message's transmission takes longer than the stated bound.

44

Programação Distribuída / José Marinho

Modelos Fundamentais

- Mascarar falhas
 - Ocultar as falhas ou convertê-las em falhas menos graves
 - Por exemplo, o campo *Frame Check Sequence* (FCS) presente nos quadros Ethernet permite transformar uma falha arbitrária noutra por omissão
- Fiabilidade de comunicação
 - Validade

Qualquer mensagem no *buffer* de saída na origem é, eventualmente, colocada no *buffer* de entrada no destino
 - Integridade

A mensagem recebida é idêntica à enviada e não existem duplicações

45

Programação Distribuída / José Marinho

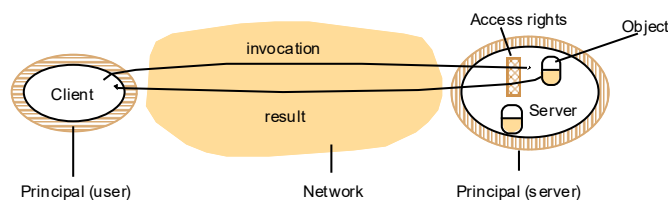
Modelos Fundamentais

- Modelo de segurança
 - Objectos acedidos por diversos utilizadores remotos
 - O mesmo objecto pode ter diversas utilizações de acordo com os respectivos utilizadores
 - Existem direitos de acesso
 - Existe o conceito de Principal
 - Entidades que invocam e fornecem resultados
 - É necessário, em cada invocação/resultado, o servidor/cliente verificar a identidade do Principal

46

Programação Distribuída / José Marinho

Modelos Fundamentais



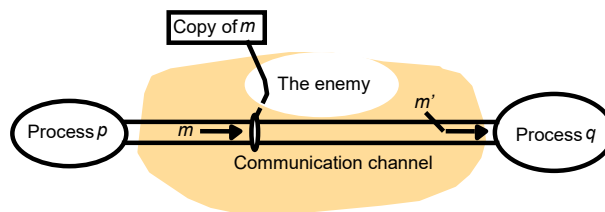
- Aspectos críticos a considerar no modelo de segurança
 - Objectos expostos através de interfaces
 - Dados expostos através do envio de mensagens

47

Programação Distribuída / José Marinho

Modelos Fundamentais

- As mensagens podem ser copiadas, modificadas e inseridas no canal de comunicação
- Exemplo: a origem de uma mensagem pode ser falsificada
- Para canais não seguros, a solução passa pela criptografia e autenticação
- Canais seguros (e.g., SSL)



48

Programação Distribuída / José Marinho

Modelos Fundamentais

- Outros tipos de ameaças
 - Fazer demasiadas invocações a um servidor (*Denial of Service Attack*)
 - Enviar demasiadas mensagens
 - Código móvel (“Cavalo de Tróia”)
- As técnicas de segurança apresentam custos de processamento e de gestão, sendo importante identificar os possíveis ataques e escolher uma solução equilibrada

49

Programação Distribuída / José Marinho